



PHASE 02 · PROCESS DOCUMENTATION

My Process:

How I built the AI Portfolio Copilot.

A transparent and honest breakdown of the design decisions, engineering trade-offs, and critical moments that shaped the development of my AI-integrated professional portfolio.

AUTHOR

Arnav Gupta

TRACK

AI Track (PS-1)

INSTITUTE

BITS Pilani Goa

DATE

June 17, 2026

1. WORK UNIT DEFINITION

What I Set Out to Build

My core objective in this work unit was to design, develop, and deploy a responsive, glassmorphic, single-page professional portfolio website featuring a hybrid server-side and client-side grounded AI Career Copilot assistant. The project addresses the challenge of presenting low-level systems engineering work (such as CUDA GEMM optimizers) alongside high-level AI applications in a cohesive, interactive medium.

Through this work unit, I established the following Atomic Values:

1. Unified Career Representation: I merged my low-level HPC parallel programming work with modern web architectures into a single obsidian-themed layout.

2. Zero-Dependency Accessibility: *I implemented local keyword-matching fallbacks to guarantee the chatbot remains usable even if serverless runtimes fail or API keys are unavailable.*

3. Research Privacy Guardrails: *I structured the profile copy and chatbot grounding dataset to showcase conceptual competence and academic achievements without exposing ongoing, proprietary research data from my work at BITS Goa's DASH Lab.*

4. Resilient Lead Capture: *I integrated direct-to-email contact form routing with failover backups to capture visitor messages reliably without requiring dedicated SMTP backend infrastructure.*

2. KNOWLEDGE CARDS

Key Insights & Lessons Learned

The following cards detail five critical decisions, debug cycles, and boundary checks from my process, detailing both what failed and what ultimately worked.

CARD 01

Architectural Hybrid AI Engine (Technical Decision)

● ATOMIC VALUE TAG

Reduced API vulnerability and minimized hosting costs.

● SITUATION / CONTEXT

I needed a conversational AI assistant on my landing page to answer recruiters' questions. However, embedding raw Gemini API keys in the client-side JavaScript exposes them to theft, while hosting a dedicated 24/7 Node.js backend server introduces hosting overhead and potential downtime.

● THE FRICTION

I faced a trade-off between user convenience and backend security. Requiring visitors to input their own API keys before chatting would ruin user experience, while forcing all queries through a dedicated web server would incur setup costs.

● WHAT FAILED

I tried obfuscating the API key in client-side code (which was easily extracted via browser inspection) and tried prompting the user to supply their own key (which created high initial user friction and drop-off).

● WHAT WORKED

I built a **Hybrid AI Engine**. By default, it operates in **Offline Mode** using a client-side keyword and Jaccard similarity matcher on the localized `portfolioData.js` object. When deployed, it queries a secure serverless endpoint proxy (`/api/chat` serverless function) to retrieve answers from Gemini 2.5 Flash using system-level environment variables, fallback-safeguarding token usage.

● THE PRINCIPLE

Progressive Degradation & Zero-Cost Serverless Proxies: Build systems that degrade gracefully to static local search heuristics when external services are offline or API keys are absent, and leverage serverless endpoints to perform secure key-dependent operations.

● WHEN THIS APPLIES

When building public portfolios or client-facing showcase websites that require LLM interactions without exposing backend resources or paying for dedicated server runtimes.

● AI PROMPTS USED

How I asked the AI for guidance on the offline fallback:

```
"How can I design a lightweight, zero-dependency keyword matcher in JavaScript that handles user search queries by calculating simple token overlap against a predefined JSON schema without using npm libraries?"
```

CARD 02

Persona-Bound Grounding Rules (Prompt Engineering)

● ATOMIC VALUE TAG

Guided recruiter engagement and restricted token usage.

● SITUATION / CONTEXT

I needed to configure the Gemini 2.5 Flash system prompt to represent me professionally and discuss **only** topics grounded in my resume data (education, CUDA projects, BITS Goa coursework).

● THE FRICTION

Out-of-the-box LLMs are generalists. If a visitor asks my portfolio chatbot to write a Python script, solve general physics problems, or write a poem, the model will comply. This consumes API quota, dilutes the professional context, and presents a branding risk.

● WHAT FAILED

Writing a basic positive system prompt: *""You are Arnav's career assistant. Use this resume data to answer questions about him.""* The model still happily answered off-topic math questions, solved generic coding exercises, and occasionally hallucinated details not found in the resume.

● WHAT WORKED

I designed a strict system instruction structure with explicit negative constraints and boundary enforcement:

1. Persona Lock: "You are strictly Arnav's Career Copilot, representing him professionally."
2. Strict Grounding: "Answer *ONLY* using the provided context. If a detail is missing, say: 'I don't have information on that in Arnav's profile records.'"
3. Unrelated Query Blocks: "If the user asks about unrelated topics (e.g. general news, coding exercises, cooking recipes, math tasks not related to Arnav's research, writing poems), say: 'I am only trained to answer questions about Arnav Gupta's professional career, skills, and background.'"

● THE PRINCIPLE

Boundary-Enforced Scoping: Grounding RAG systems requires defining strict negative boundaries (what the model **must not** answer) rather than only specifying what it **should** reference.

● WHEN THIS APPLIES

When deploying user-facing AI chat widgets where query scopes must be strictly controlled to prevent misuse or brand dilution.

● AI PROMPTS USED

How I designed the constraint prompt:

```
"Refactor this system prompt to enforce strict negative guardrails on Gemini 2.5 Flash so that it rejects any queries that are not directly related to the candidate's resume, without
```

sounding overly robotic or hostile to recruiters."

CARD 03

Third-Party Form Handshakes (Debugging / What Broke)

● ATOMIC VALUE TAG

Confirmed client lead delivery and resolved silent failures.

● SITUATION / CONTEXT

I hooked up the "Get in Touch" contact form to forward visitor messages directly to my BITS Pilani email ('f20240843@goa.bits-pilani.ac.in').

● THE FRICTION

In local development, the contact form simulated a success state, but on the live Vercel site, submissions failed to deliver or ended up silent because form processors (like FormSubmit.co) check requesting domains and origins to prevent spam and verify ownership.

● WHAT FAILED

Submitting form data via AJAX without specifying the proper HTTP headers, resulting in silent spam blocks or CORS rejections on the production site, while the local fallback incorrectly assumed delivery.

● WHAT WORKED

I refactored the backend API request to explicitly pass domain origin parameters ('Origin' and 'Referer' headers) to mock the verified domain handshake, and implemented a serverless Node.js route '/api/contact' that handles environment validation and returns a clean 200 fallback when writing backup logs to a read-only serverless disk fails.

● THE PRINCIPLE

Environment-Specific Handshake Verification: Zero-backend third-party integrations (like email submitters) must be audited across different origin states (localhost vs. production domains) to verify CORS and origin filtering rules.

● WHEN THIS APPLIES

When building serverless or static portfolios that require contact forms with automated email delivery without dedicated SMTP backends.

● AI PROMPTS USED

How I debugged the CORS blockage:

"Why does FormSubmit.co return a CORS error or fail to deliver when called via an AJAX fetch in a Node.js Express server acting as a proxy, and how do I configure the Referer and Origin headers to bypass this?"

CARD 04

Visual Space Hierarchy (UI/UX Refactoring)

● ATOMIC VALUE TAG

Improved layout readability and restored viewport constraints.

● SITUATION / CONTEXT

Designing the layout for my resume and HPC project cards. My initial design used a 50/50 split-screen layout (portfolio on the left, chat box on the right).

● THE FRICTION

On standard viewports (especially laptops), both columns competed for width. This caused structural squishing: text overflowed, scrollbars appeared inside tiny panels, and critical elements (like BITS Pilani tags) were cropped. The main page scrollbar was stuck in the middle of the screen instead of the edge.

● WHAT FAILED

Attempting to use CSS media queries to resize the split panes on medium screens, which still looked extremely cramped and squashed the resume content.

● WHAT WORKED

I changed the layout. The main portfolio content was refactored into a single full-width column centered on the screen, letting it scroll naturally with the browser's scrollbar on the far right. The chat box was rebuilt as a floating, collapsible widget in the bottom-right corner that can be opened or closed at will.

● THE PRINCIPLE

Visual Layout Dominance: Primary content should occupy the center stage, while conversational assistants and conversational interfaces should float on a separate presentation layer to prevent layout competition.

● WHEN THIS APPLIES

When designing dashboards or information-heavy layouts that feature complementary AI chat assistance.

● AI PROMPTS USED

How I refactored the layout structure:

```
"How can I transition a two-column split-pane web app where the right pane is a chat interface into a single-column layout with a floating, collapsible chat widget in the bottom-right corner using CSS flex/grid and fixed positioning?"
```

CARD 05

Research Discretion & Intellectual Property (Information Privacy)

● ATOMIC VALUE TAG

Protected research confidentiality while demonstrating technical competency.

● SITUATION / CONTEXT

Displaying my research on LLM inference systems at BITS Goa's DASH Lab.

● THE FRICTION

Presenting impressive metrics (KV cache hit rates, predictive caching policies) while keeping academic research confidential. Putting too many specific metrics or live charts on the portfolio could expose intellectual property.

● WHAT FAILED

Designing an interactive KV Cache simulator widget on the website with live hit-rate charts. This made the implementation details too specific, potentially exposing ongoing academic study.

● WHAT WORKED

I stripped the interactive caching metrics charts out of the UI. I redesigned the copy to focus on high-level learnings: LLM inference architectures, and simulation workload concepts for cost-effective systems.

● THE PRINCIPLE

Conceptual Competence over Metric Disclosure: In portfolios showcasing ongoing research, highlight methodological expertise and architectural understanding rather than specific performance numbers or implementation details.

● WHEN THIS APPLIES

When building portfolios while working inside academic labs or industry settings with strict IP or NDA guidelines.

● AI PROMPTS USED

How I reworded the research description to protect IP:

"Rewrite a research description about optimizing KV caches using reinforcement learning to be high-level and conceptual, focusing on learning Large Language Model architectures and simulation workloads rather than disclosing performance metrics or experimental results."